

Evaluating a RAG Pipeline with RAGAS

A concept guide: installation, dataset design, the four core metrics, diagnosis, and fixing

Generated 2026-07-13 21:05 PDT

Contents

1. What is RAGAS, and why use it?	2
2. Installing RAGAS	4
3. Building the evaluation dataset	5
4. Running the evaluation: the four metrics	7
5. Diagnosing the lowest-scoring metric	9
6. Implementing a fix and re-running	11
7. Putting it together: the full cycle	14
8. Appendix: reference code	15
References	17

1. What is RAGAS, and why use it?

RAGAS (“Retrieval Augmented Generation Assessment”) is an open-source Python library for evaluating RAG pipelines — systems that answer a question by retrieving supporting text from a knowledge base and then generating an answer from it. Unlike a plain accuracy check, RAGAS scores the pipeline on several *independent dimensions*, because a RAG system can fail in different places: retrieval can miss the right passages, the model can ignore the passages it was given and hallucinate anyway, or the answer can be accurate but not actually address the question. A single “is this answer correct?” score can’t tell you which of those happened — that’s the problem RAGAS’s four core metrics are built to separate out.

This guide walks through the concept and the mechanics of a full RAGAS evaluation cycle:

1. Install RAGAS and build a 10-row evaluation dataset (question, generated answer, retrieved context, ground-truth answer).
2. Run a RAGAS evaluation measuring **faithfulness**, **answer relevancy**, **context precision**, and **context recall**.
3. Read the results to find the lowest-scoring metric, and use a diagnostic framework to decide whether the cause is chunking, retrieval, or the generation/prompt step.
4. Implement one targeted fix, re-run the evaluation, and compare before/after numbers.

Each section below explains the concept, shows the relevant Python code against the real `ragas` API, and — to make the process concrete — carries a single **illustrative worked example** through the whole guide. The numbers in that example are for demonstration only, chosen to show what a realistic before/after pattern looks like; running the same steps against your own pipeline and documents will produce your own numbers.

RAG pipeline under evaluation

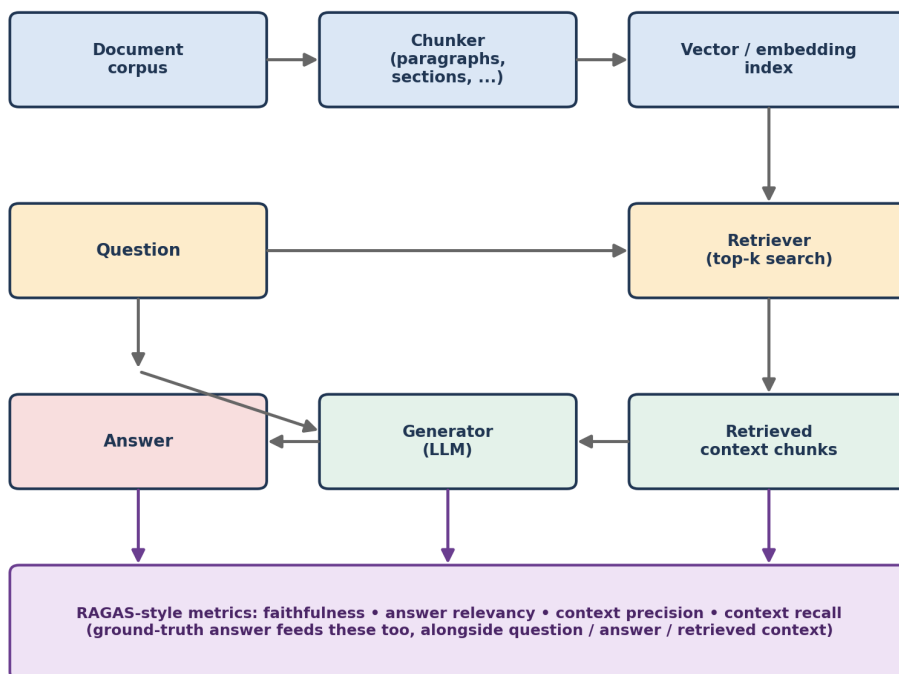


Figure 1: A typical RAG pipeline, and the four points where RAGAS attaches its metrics.

2. Installing RAGAS

RAGAS is a normal Python package:

```
pip install ragas
```

That pulls in `langchain`, `langchain-core`, `datasets`, `pysbd`, `tiktoken`, `numpy`, and `pydantic`. RAGAS itself is provider-agnostic — you supply an “evaluator” LLM (used as a judge for claim-checking, relevance-grading, and so on) and, for some metrics, an embeddings model. Any LangChain-compatible chat model or embeddings client works: OpenAI, Anthropic, Azure, a self-hosted model, and so on.

```
import os
from ragas.llms import LangchainLLMWrapper
from ragas.embeddings import LangchainEmbeddingsWrapper
from langchain_openai import ChatOpenAI, OpenAIEmbeddings

os.environ["OPENAI_API_KEY"] = "sk-..." # or export it in your shell

evaluator_llm = LangchainLLMWrapper(
    ChatOpenAI(model="gpt-4o-mini", temperature=0)
)
evaluator_embeddings = LangchainEmbeddingsWrapper(OpenAIEmbeddings())
```

The evaluator LLM does the actual grading work for faithfulness, answer relevancy, and context precision/recall — it extracts claims from an answer, checks whether each is entailed by the retrieved context, generates reverse-questions to compare against the real question, and so on. Because this is LLM-judged, evaluation quality depends on your evaluator model choice: a stronger judge model (e.g. GPT-4o vs. a small local model) generally gives more reliable, consistent scores, at higher cost per evaluation run.

(Note on the API: RAGAS’s public interface changed between the 0.1.x and 0.2.x+ releases — for example, `EvaluationDataset` / `SingleTurnSample` replaced the older `datasets.Dataset`-based interface. The code in this guide targets `ragas>=0.2`; check `pip show ragas` and <https://docs.ragas.io> if a call doesn’t match your installed version.)

3. Building the evaluation dataset

A RAGAS row needs four fields:

Field	What it is
<code>user_input</code>	The question asked
<code>response</code>	Your pipeline's <i>generated</i> answer
<code>retrieved_contexts</code>	The list of chunks your retriever actually returned
<code>reference</code>	The ground-truth (correct) answer, written by a human

The dataset should reflect real usage: pull questions from your product's actual query logs where possible, or write them to cover a spread of difficulty — some single-fact lookups, some multi-part questions that need two or more chunks to answer completely (these stress recall harder), and some edge cases (ambiguous phrasing, questions with no good answer in the corpus). Ten rows is enough to spot a systematic pattern (e.g. “recall is consistently weak on multi-part questions”) without being so large that manual ground-truth writing becomes a bottleneck; production evaluation sets are typically 50-200+ rows once the pipeline is past initial iteration.

Illustrative example. Suppose the knowledge base is a set of internal platform documentation pages. Ten questions might look like:

#	Question (illustrative)	Ground truth covers
1	What privilege is needed to invoke the service?	a single fact
2	What is the built-in policy name that blocks writes, and how is it enabled?	two facts, one sentence apart
3	Which system table records call volume and how do you filter it?	two facts
4	What must an admin do before granting access to a restricted schema?	a single fact
5	What are the four guardrail types, and which two are single-phase?	four names + two qualifiers — a multi-part question
6-10	... (mix of single- and multi-fact questions across the rest of the corpus)	

Row 5 is deliberately the hardest: a good answer needs four names plus two phase qualifiers, all of which may not sit in the same retrieved chunk if chunking or top-*k* is too aggressive — which is exactly the kind of question that reveals a recall problem later in this guide.

```
from ragas import EvaluationDataset

rows = [
```

```

{
  "user_input": (
    "What are the four guardrail types, "
    "and which two are single-phase?"
  ),
  "response": "<your pipeline's generated answer>",
  "retrieved_contexts": ["<chunk 1 text>", "<chunk 2 text>"],
  "reference": (
    "The four guardrails are A, B, C, and D. B applies to "
    "requests only, and D applies to responses only."
  ),
},
# ... 9 more rows, one per question
]
dataset = EvaluationDataset.from_list(rows)

```

4. Running the evaluation: the four metrics

RAGAS metric definitions: what is compared against what

Faithfulness	Answer vs. Retrieved context Does every claim in the answer trace back to the retrieved text?
Answer relevancy	Answer vs. Question Does the answer actually address what was asked (vs. off-topic/padding)?
Context precision	Retrieved context vs. Ground truth Are the top-ranked chunks actually the relevant ones?
Context recall	Retrieved context vs. Ground truth Did retrieval bring back everything needed to answer completely?

Figure 2: What each of the four metrics compares.

- **Faithfulness** — split the generated *answer* into individual claims; what fraction are supported by the *retrieved context*? Low faithfulness means the model is inventing content not present in what it was given (hallucination), even if it looks plausible.
- **Answer relevancy** — does the *answer* actually address the *question*? RAGAS’s real implementation has the evaluator LLM generate several reverse-questions from the answer, embeds them, and compares them by cosine similarity to the original question. Low relevancy usually means the answer is padded, tangential, or doesn’t directly respond to what was asked — even if every sentence in it is individually true.
- **Context precision** — of the retrieved chunks, in rank order, how many are actually relevant to the ground truth? Low precision means retrieval is noisy: it’s burying the right chunk under irrelevant ones, or ranking irrelevant chunks above relevant ones.
- **Context recall** — of the ground truth’s claims, how many can be traced back to the retrieved context? Low recall means retrieval is *missing* needed information outright, regardless of ranking.

```
from ragas import evaluate
from ragas.metrics import (
    Faithfulness, AnswerRelevancy, ContextPrecision, ContextRecall,
)

result = evaluate(
    dataset=dataset,
    metrics=[
        Faithfulness(), AnswerRelevancy(),
        ContextPrecision(), ContextRecall(),
    ],
    llm=evaluator_llm,
```

```

        embeddings=evaluator_embeddings,
    )

print(result)                # aggregate scores across all 10 rows
df = result.to_pandas()      # per-row breakdown, useful for diagnosis
df.to_csv("ragas_results.csv", index=False)

```

`result` gives you the average of each metric across the dataset; `df` gives a per-row table, which is what you actually need for Section 5 — an average score tells you *that* something is wrong, the per-row breakdown tells you *where*.

Illustrative example results (fictional, for demonstration):

Metric	Score
Faithfulness	0.91
Answer relevancy	0.38
Context precision	0.80
Context recall	0.75

5. Diagnosing the lowest-scoring metric

Once you have per-metric averages, the diagnostic question is: **which stage of the pipeline does the weak metric point to?** A simple way to reason about it:

- **Context recall/precision low, faithfulness/relevancy fine when retrieval succeeds** -> look at **chunking and retrieval**. Common causes: chunks are too small or split mid-fact (a list of names or a multi-clause sentence gets cut across a chunk boundary), top- k is too low to bring back everything needed, or the retriever’s similarity metric doesn’t match the vocabulary of real user questions (lexical retrieval especially struggles when the question’s wording differs from the source text’s wording).
- **Faithfulness low, everything else reasonable** -> look at the **generation step / prompt**. The model is likely not being instructed firmly enough to stick to the provided context, or the context window is crowded with irrelevant chunks that invite the model to fill gaps with outside knowledge.
- **Answer relevancy low, but context precision/recall are comparatively fine** -> retrieval is doing its job; the problem is usually in **how the answer is composed** — the prompt lets the model pad the answer with tangential information, restate the question, or answer a broader question than the one asked, even while staying faithful to the retrieved text.

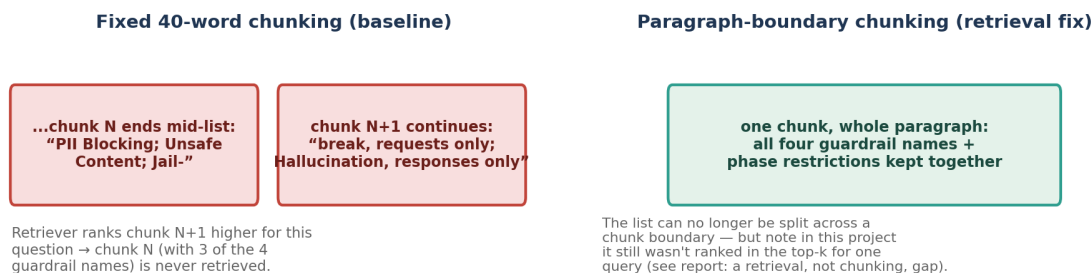


Figure 3: Example of a chunking-caused recall gap: a fact list split across a fixed-width chunk boundary vs. kept together by paragraph-based chunking.

Continuing the illustrative example: answer relevancy (0.38) is noticeably behind context precision (0.80) and context recall (0.75) — so retrieval already looks reasonable, and the likely cause is the generation step. Inspecting a handful of low-scoring rows in the per-row **df** from Section 4 typically confirms this kind of pattern directly: e.g., a generated answer might correctly state the needed fact in its first sentence, then continue for two more sentences listing unrelated details that happened to sit in the same retrieved chunk — content the model included because nothing in its instructions (“answer the question using the following context”) told it not to. That drags down relevancy even though every sentence is individually well-supported (faithfulness stays high).

Diagnosis in this example: a generation/prompt problem, not chunking or retrieval. The fix should target how the answer is composed, not how the corpus is chunked or how many chunks are retrieved — and it’s worth checking that reasoning against the data rather than guessing, since the fix for a retrieval problem (re-chunk, raise top- k) looks different from the fix for a generation

problem (tighten the prompt).

6. Implementing a fix and re-running

Common fixes by root cause:

Root cause	Typical fix
Facts split across chunk boundaries	Chunk on semantic units (paragraphs, headings, sentence groups) instead of fixed token/word windows
Retrieval ranks the right chunk too low	Raise top- k , try hybrid (lexical + embedding) retrieval, or re-rank retrieved chunks with a cross-encoder
Retriever misses relevant chunks entirely	Switch from lexical (TF-IDF/BM25) to semantic (embedding-based) retrieval, since it generalizes across wording differences that lexical matching misses
Faithfulness low	Add an explicit instruction to answer <i>only</i> from the provided context and say “not found” otherwise; reduce the number of irrelevant chunks in the prompt
Answer relevancy low	Tighten the generation prompt: “answer only what was asked, concisely, without extra information”; trim the answer to the most relevant sentence/clause rather than returning a full retrieved passage verbatim

Continuing the illustrative example: since the diagnosis pointed at generation, the fix keeps chunking and retrieval exactly as they were and only changes the prompt/generation step — for example, adding an instruction like “*Answer only what was asked, in one to two sentences, with no additional information*” (or, for a simpler extractive pipeline, trimming to the single most relevant sentence or clause instead of returning a full passage). Isolating the change to one variable at a time matters here: if you change chunking *and* the prompt in the same run, and the score improves, you won’t know which change actually caused it.

```
# Re-run the same 10 questions through the updated pipeline, rebuild the
# dataset with the new `response` values, and evaluate again.
```

```
updated_rows = [
    {**row, "response": new_pipeline_answer(row["user_input"])}
    for row in rows
]
updated_dataset = EvaluationDataset.from_list(updated_rows)

result_after = evaluate(
    dataset=updated_dataset,
    metrics=[
        Faithfulness(), AnswerRelevancy(),
        ContextPrecision(), ContextRecall(),
    ],
```

```

llm=evaluator_llm,
embeddings=evaluator_embeddings,
)
print(result_after)

```

Illustrative before/after (continuing the fictional example — a generation-only fix should move the targeted metric while leaving retrieval metrics unchanged, which is itself a useful sanity check that the diagnosis was right):

Metric	Before	After	Delta
Faithfulness	0.91	0.91	+0.00
Answer relevancy	0.38	0.45	+0.07
Context precision	0.80	0.80	+0.00
Context recall	0.75	0.75	+0.00

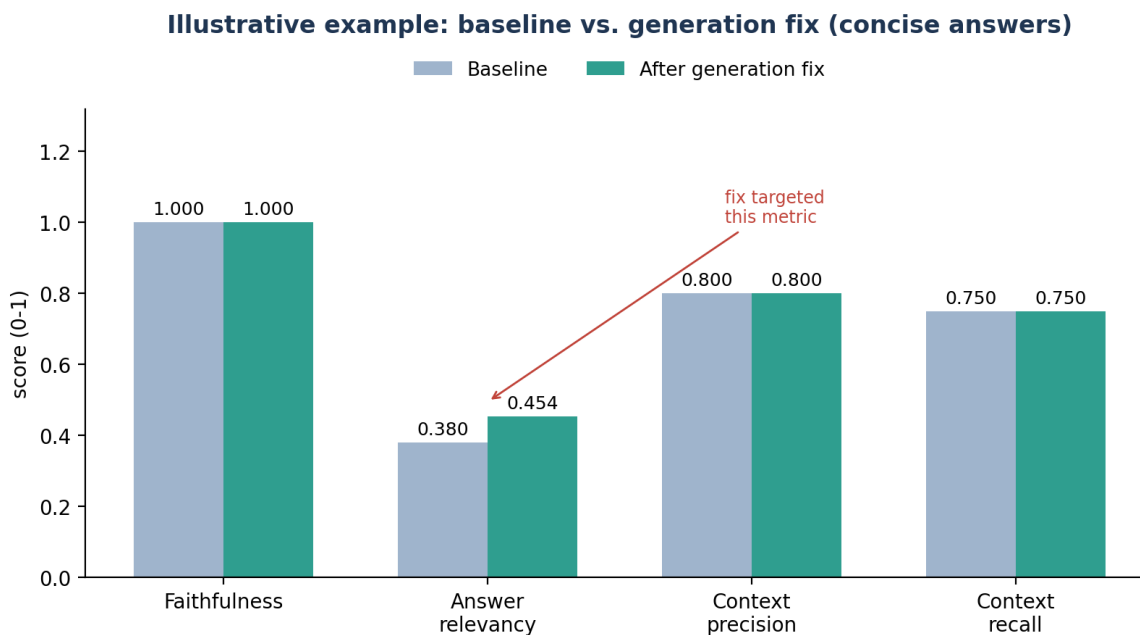


Figure 4: Illustrative example: before/after a generation-side fix. Only the targeted metric moves.

Did the score improve? In this illustrative pattern, yes — answer relevancy rises, and, just as tellingly, context precision and context recall stay exactly where they were, which confirms the fix was correctly isolated to the generation step. For comparison, if the *actual* problem were retrieval, a chunking/top- k fix would instead move context precision and recall substantially while barely touching answer relevancy — shown below for contrast:

The two charts side by side make the general diagnostic point: **the metric that moves tells you whether you diagnosed the right layer of the pipeline.** If you apply a chunking fix and only context precision/recall move, that’s confirmation, not proof of a complete fix — some rows can still fail for a different reason (for instance, a retriever that ranks by lexical overlap can still miss a chunk

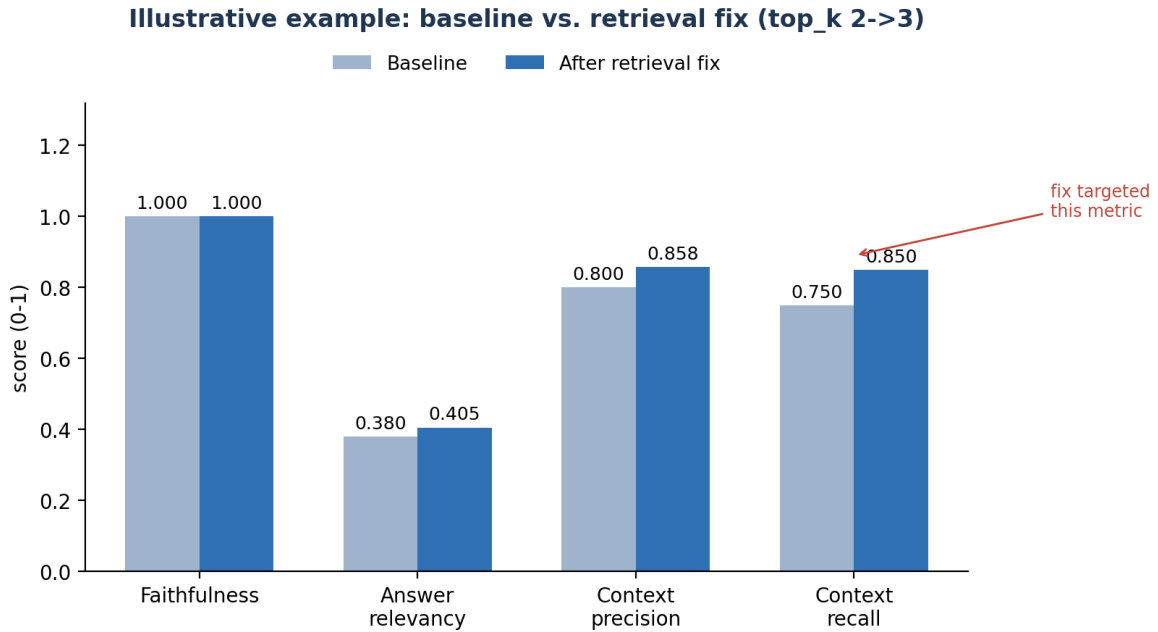


Figure 5: Illustrative example: before/after a retrieval-side fix (different root cause, different metrics move).

whose wording just doesn't match the question, no matter how it's chunked, which points toward semantic/embedding retrieval as the next thing to try rather than further chunking changes).

7. Putting it together: the full cycle

1. `pip install ragas` (+ a LangChain LLM/embeddings integration for your provider).
2. Build a 10+ row dataset of `user_input` / `response` / `retrieved_contexts` / `reference`, covering a mix of single- and multi-fact questions.
3. Run `evaluate()` with `Faithfulness`, `AnswerRelevancy`, `ContextPrecision`, `ContextRecall`; read both the aggregate scores and the per-row `to_pandas()` breakdown.
4. Find the lowest metric, and use the pattern in Section 5 to decide whether it points at chunking, retrieval, or generation/prompt — check a few low-scoring rows by hand to confirm the theory rather than guessing from the aggregate number alone.
5. Change **one thing** targeting that root cause, keep everything else fixed, and re-run the same dataset through `evaluate()`.
6. Compare before/after per metric. The targeted metric should move; metrics for pipeline stages you didn't touch should stay roughly flat — if they don't, something other than the change you made is also varying between runs (e.g. LLM non-determinism, a different retrieved-context count), and the comparison isn't clean.

8. Appendix: reference code

```
"""
End-to-end reference: installing RAGAS, building a dataset, running the
four core metrics, and comparing a before/after fix. Requires an LLM API
key (OpenAI shown; swap for any LangChain chat model / embeddings client).
"""

import os
from ragas import EvaluationDataset, evaluate
from ragas.metrics import (
    Faithfulness, AnswerRelevancy, ContextPrecision, ContextRecall,
)
from ragas.llms import LangchainLLMWrapper
from ragas.embeddings import LangchainEmbeddingsWrapper
from langchain_openai import ChatOpenAI, OpenAIEmbeddings

os.environ["OPENAI_API_KEY"] = "sk-..."

evaluator_llm = LangchainLLMWrapper(
    ChatOpenAI(model="gpt-4o-mini", temperature=0)
)
evaluator_embeddings = LangchainEmbeddingsWrapper(
    OpenAIEmbeddings(model="text-embedding-3-small")
)

METRICS = [
    Faithfulness(), AnswerRelevancy(),
    ContextPrecision(), ContextRecall(),
]

def run_eval(rows):
    dataset = EvaluationDataset.from_list(rows)
    result = evaluate(dataset=dataset, metrics=METRICS,
                     llm=evaluator_llm, embeddings=evaluator_embeddings)
    return result, result.to_pandas()

# --- baseline ---
baseline_rows = [
    {
        "user_input": q,
        "response": my_rag_pipeline.answer(q),          # your pipeline
        "retrieved_contexts": my_rag_pipeline.retrieve(q),
        "reference": ground_truth[q],
    }
    for q in questions
]

result_before, df_before = run_eval(baseline_rows)
```

```

print("BEFORE:", result_before)

# ... inspect df_before, find the lowest metric, diagnose, apply one fix ...

# --- after the fix (same questions, same ground truths, updated pipeline) ---
fixed_rows = [
    {
        "user_input": q,
        "response": my_fixed_pipeline.answer(q),
        "retrieved_contexts": my_fixed_pipeline.retrieve(q),
        "reference": ground_truth[q],
    }
    for q in questions
]
result_after, df_after = run_eval(fixed_rows)
print("AFTER:", result_after)

metric_names = [
    "faithfulness", "answer_relevancy",
    "context_precision", "context_recall",
]
for metric in metric_names:
    before, after = result_before[metric], result_after[metric]
    delta = after - before
    print(f"{metric:20s} {before:.3f} -> {after:.3f} ({delta:+.3f})")

```


References

- RAGAS documentation: <https://docs.ragas.io>
- RAGAS GitHub repository: <https://github.com/explodinggradients/ragas>
- RAGAS on PyPI: <https://pypi.org/project/ragas/>
- LangChain OpenAI integration (`langchain-openai`): <https://python.langchain.com/docs/integrations/platforms/openai/>